Assignment 2: HTTP APIs, gRPC and Metrics

Enterprise Software Development, Fall 2026

Project: Paws & Cart (extends Assignment 1)

October 25, 2026

Contents

1 Part A: Improve the HTTP API **2**

1.1 REST principles . . . 2

1.2 API before and after: placing an order . . . 2

1.3 Error format and request IDs . . . 3

1.4 Endpoint tests: valid, invalid and repeated requests . . . 4

1.5 OpenAPI specification . . . 4

1.6 Idempotency (safe repeats) . . . 5

1.7 Rate limiting . . . 5

1.8 Reliable Python caller . . . 6

1.9 Automated test suite . . . 8

2 Part B: gRPC service **8**

2.1 gRPC contract and code generation . . . 8

2.2 gRPC server, shared rules and client . . . 9

2.3 Call paths . . . 9

2.4 Success and failure through both versions . . . 10

3 Part C: Metrics **11**

3.1 Audit of the Assignment 1 metrics . . . 11

3.2 New and changed metrics . . . 12

3.3 Queries . . . 12

3.4 Design choices . . . 13

3.5 Prometheus and Grafana results . . . 13

4 Limitations and design choices **16**

1 Part A: Improve the HTTP API

1.1 REST principles

The table checks the six REST constraints against the Paws & Cart API as it stood at the end of Assignment 1. Code on demand is the only optional constraint.

Constraint	Meaning	Shown?	Example from Paws & Cart
Client–server	The user's program is separate from the program that stores and processes the data; they communicate only through the API.	Yes	The React page, the Python caller and curl are all clients. None touch SQLite; they only send HTTP requests such as `POST /orders`.
Stateless	The server remembers nothing between requests; each request carries all it needs.	Yes	There are no logins, cookies or sessions. An order request names the product and quantity in the body; the server needs no earlier request.
Cacheable	Responses say whether they may be stored and reused.	No	The API sends no `Cache-Control` or `ETag`. Stock changes with every order, so most responses should not be reused anyway.
Layered system	Proxies or gateways can sit between client and server unnoticed.	Yes	nginx sits in front of Flask; the browser talks only to nginx on port 5173, and nginx forwards `X-Real-IP` so the rate limiter still sees the real client.
Uniform interface	Every resource has its own URI; clients use the same standard methods, status codes and data format.	Partly	Products and orders have their own URIs, responses are JSON, errors use proper status codes. But completing an order is a verb in the path (`POST /orders/{id}/complete`) and responses carry no links to next actions (HATEOAS).
Code on demand (optional)	The server can send code that the client runs.	Yes	The nginx container serves the React JavaScript bundle that the browser runs to call the API.

1.2 API before and after: placing an order

The chosen user task is *placing an order*. The endpoint keeps its shape (`POST /orders`); what changed is what a repeated or excessive request does.

	Before (Assignment 1)	After (Assignment 2)
Endpoint	`POST /orders` with `product_id` and `quantity`.	Same endpoint and body, plus an optional `Idempotency-Key` header.
Problem	A double click or a client retry after a lost response created two orders and took stock twice. There was no limit on how fast one client could order, and no defined message for a blocked client.	—
Change	—	The key and a hash of the body are stored with the order in one transaction. A repeat with the same key returns the original order with `Idempotent-Replayed: true`; the same key with a different body returns 422. More than 10 orders per 60 seconds from one client IP returns 429 with a `Retry-After` header.
Reason	—	Placing an order is payment-like: it must be safe to retry. A limit protects the server from floods, including floods made of retries. Both use standard HTTP behaviour that any caller can understand.

Other endpoints: changed or left alone.

Endpoint	What happened	Why
`GET /health`	Left alone	Simple health check.
`GET /metrics`	Changed slightly	Still the Prometheus endpoint, but it now recomputes the pending-orders gauge from SQLite on every scrape, because the gRPC service can change orders too.
`GET /products`	Left alone	Already correct.
`GET /orders`	Left alone	Already correct.
`POST /orders/{id}/complete`	Left alone	Not repeat-safe by design (a second call returns 409), so the Python caller never retries it.

Callers updated. The React page (`frontend/src/App.jsx`) creates a new idempotency key for every Buy click, retries once on a network failure with the *same* key, and shows a friendly message built from `Retry-After` on a 429 (Figure 2). A new Python caller (`client/order_client.py`, Section 1.8) sends a key with every order.

1.3 Error format and request IDs

Every error response is a JSON object with an `error` message that a person can read; some add fields a program can use:

```
400  {"error": "quantity must be a positive integer."}
404  {"error": "Product not found."}
409  {"error": "Not enough stock available.", "available_stock": 1}
422  {"error": "Idempotency-Key was already used with a different request."}
429  {"error": "Too many requests.", "limit": 10, "window_seconds": 60, "retry_after_seconds": 54}
```

Every response, successful or not, also carries an `X-Request-ID` header. The server reuses an

incoming ID or creates a new one, and writes it into its JSON log line, so one request can be found in Kibana. The 429 example values are illustrative; the field names come from `app.py`.

1.4 Endpoint tests: valid, invalid and repeated requests

The 27 REST tests in `tests/test_openapi_conformance.py` run against a throwaway SQLite database. The main cases for `POST /orders`:

Test	Request	Result
Valid request	`product_id 1, quantity 2`	201; body matches the `Order` schema; status `pending`.
Repeated request	The same body and the same `Idempotency-Key` again	201 both times, identical order; only the second has `Idempotent-Replayed: true`; stock drops once; exactly one order exists.
Same key, other body	Same key, quantity changed	422.
Failed request retried	409 (quantity 9999), then a valid body with the same key	201: a failed request does not use up its key.
Invalid: zero quantity	`quantity 0`	400 with the `Error` schema.
Invalid: wrong type	`product_id: true`	400 (booleans are rejected explicitly).
Invalid: missing field	no `quantity`, or no `product_id`	400.
Invalid: not JSON	plain-text body	400.
Invalid: long key	200-character key	400.
Unknown product	`product_id 999`	404.
Not enough stock	`quantity 9999`	409 with the `InsufficientStock` schema.
Rate limit	11th request in 60 s	429, `Retry-After` between 1 and 60 s, no order created.

1.5 OpenAPI specification

Purpose. OpenAPI is a standard, machine-readable format (JSON or YAML) for describing an HTTP API: its endpoints, the inputs each accepts, and every success and error response. It is a contract between the API and its callers, so people and tools can use the API without reading its code.

How I created it. `openapi.yaml` (OpenAPI 3.0.3) was written by hand for the Flask API, since Flask does not generate a spec from code. It describes all six endpoints, reusable schemas (`Product`, `Order`, `CreateOrderRequest`, `Error`, `InsufficientStock`, `RateLimited`), the `Idempotency-Key` parameter, and the response headers `X-Request-ID` and `Idempotent-Replayed`. Every response a request can produce, including 400, 404, 409, 422 and 429, is listed.

How I used it. A hand-written spec can drift from the code, so the tests make it enforceable: `test_spec_is_valid_openapi` validates the document with `openapi-spec-validator`, and the other tests call the real Flask app and validate each response body against the schema the spec names, using `jsonschema`. A missing field, wrong type or undocumented status code fails a test. Excerpt of the order endpoint:

```
/orders:
  post:
    summary: Create a pending order and reduce stock
    operationId: createOrder
    parameters:
      - $ref: "#/components/parameters/IdempotencyKey"
    requestBody:
      required: true
```

```
    content:
      application/json:
        schema: { $ref: "#/components/schemas/CreateOrderRequest" }
  responses:
    "201": order created, or the original order on a replay
    "400": invalid input        "404": unknown product
    "409": not enough stock     "422": key reused with another body
    "429": rate limited (Retry-After header, RateLimited schema)
```

1.6 Idempotency (safe repeats)

A request may be sent more than once, for example when a caller times out and retries although the first request succeeded. `POST /orders` is not naturally safe to repeat, because each call creates an order.

Design. The optional `Idempotency-Key` header (1–128 characters) identifies one logical order.

- The server hashes the request body with SHA-256 and checks a new table, `idempotency_keys` (key, request hash, order id, created time), inside the same `BEGIN IMMEDIATE` transaction that creates the order. The transaction takes the write lock, so two simultaneous retries cannot both create an order.
- Same key and same body: the original order is returned with status 201 and `Idempotent-Replayed: true`. Stock is not changed again.
- Same key and a different body: 422, because a key belongs to one specific request.
- The key is stored only when the order is actually created, so a request that fails (for example 409) can be retried with the same key after restocking.
- Without a key, the endpoint behaves exactly as in Assignment 1.

1.7 Rate limiting

`POST /orders` is limited by a sliding-window limiter inside the Flask app (single instance, counts kept in memory).

- **Limit:** 10 requests per 60-second window (configurable with `RATE_LIMIT_MAX` and `RATE_LIMIT_WINDOW_SECONDS`).
- **Who shares it:** each client IP address has its own allowance, taken from the `X-Real-IP` header set by nginx. Everyone behind the same IP shares one counter, and failed requests (such as 409) count too.
- **When calls are allowed again:** the window slides, so a request is allowed again once the oldest counted request is 60 seconds old. A rejected request gets 429 with a `Retry-After` header giving that number of seconds, and a JSON body with the limit and window.

The check runs before validation and database work, so a blocked request costs almost nothing and never creates an order. Rejections are visible in Prometheus as `status_code="429"` on the HTTP duration histogram.

Test result. `client/demo_rate_limit.py` picks the best-stocked product and places orders quickly: orders 1–10 return 201 and the 11th returns 429 with `Retry-After` (Figure 1). The browser shows the same limit as a message to the customer (Figure 2). The automated tests also prove that the limit is per IP, applies only to `POST /orders`, creates no order when it blocks, and lets requests through again after the window.

```
PS C:\Users\Pc\petstore-observability> docker run --rm --add-host=host.docker.internal:host-gateway -v "${PWD}:/app" -w /app python:3.12-slim python
client/demo_rate_limit.py http://host.docker.internal:5000
request  1: HTTP 201
request  2: HTTP 201
request  3: HTTP 201
request  4: HTTP 201
request  5: HTTP 201
request  6: HTTP 201
request  7: HTTP 201
request  8: HTTP 201
request  9: HTTP 201
request 10: HTTP 201
request 11: HTTP 429  Retry-After=60s  body={'error': 'Too many requests.', 'limit': 10, 'retry_after_seconds': 60, 'window_seconds': 60}
request 12: HTTP 429  Retry-After=60s  body={'error': 'Too many requests.', 'limit': 10, 'retry_after_seconds': 60, 'window_seconds': 60}

waiting 60s as the server asked ...
after waiting: HTTP 201
```

Figure 1: Rate-limit demo: ten orders accepted (201), then 429 with `Retry-After`.

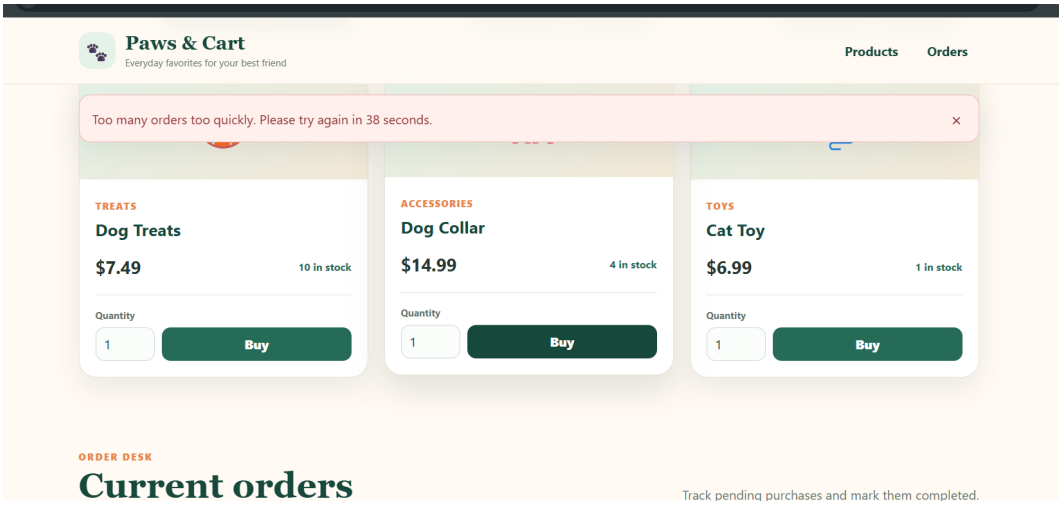


Figure 2: The React page shows a friendly message built from the `Retry-After` header.

1.8 Reliable Python caller

`client/order_client.py` acts as another service calling the API, using only the Python standard library. It adds three protections:

- **Timeout:** each attempt is abandoned after 2 seconds, so a hung server cannot hang the caller.
- **Limited retry with backoff:** a timeout, connection error or 5xx is retried, at most 3 attempts per call, waiting 0.5 s then 1 s (doubling, capped at 4 s, multiplied by random jitter so many clients do not retry in step). Only safe operations are retried: `GET` requests, and `POST /orders` because every attempt of one logical order sends the same `Idempotency-Key`. `POST /orders/{id}/complete` is never retried. A 429 makes the caller wait the server's `Retry-After`, and does not count as a failure.
- **Circuit breaker:** after 3 failed calls in a row the breaker opens and calls fail immediately for 10 seconds without touching the server; then one trial call is let through (half-open). Success closes the breaker, failure opens it for another period (Figure 3).

Setting	Value
Timeout	2 s per attempt (demo: 0.3 s)
Retry limit	3 attempts in total, transient errors only
Backoff	0.5 s, doubling, jitter, capped at 4 s (demo: 0.2 s, capped at 1 s)
Breaker failure threshold	3 failed calls in a row
Breaker recovery period	10 s, then one trial request (demo: 3 s)

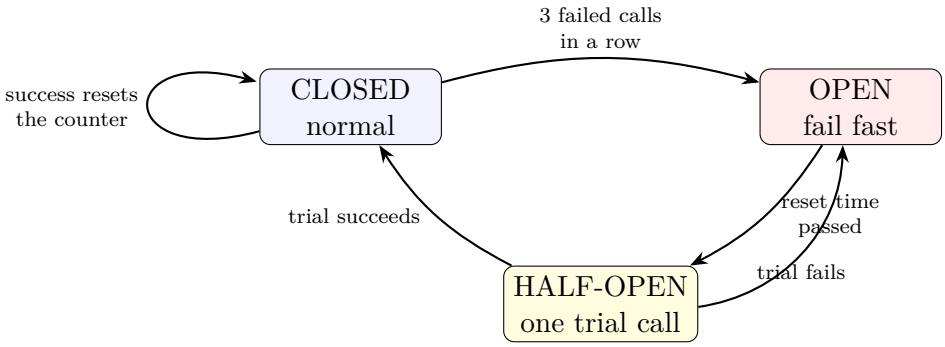


Figure 3: The circuit breaker as a state machine.

Test method. `client/demo_reliability.py` runs the caller against `client/fake_server.py`, a small controllable server (modes: ok, error, slow, limited), so every failure can be produced on demand. The demo uses short settings (timeout 0.3 s, backoff 0.2 s capped at 1 s, reset 3 s) so it finishes quickly. Output of one run:

```
SETTINGS: timeout=0.3s, max_attempts=3, backoff=0.2s*2^n with jitter (cap 1s), breaker opens after 3 failed calls, reset after 3s

1) SUCCESS
   -> HTTP 201, order #1

2) RETRY after server errors (first 2 requests return 503, third works)
[client] POST /orders attempt 1/3 failed (HTTP 503); retrying in 0.11s
[client] POST /orders attempt 2/3 failed (HTTP 503); retrying in 0.36s
   -> HTTP 201, order #2; server saw 1 order for this key: True

3) RETRY after a TIMEOUT (first request is slower than the 0.3s timeout)
[client] POST /orders attempt 1/3 failed (TimeoutError: timed out); retrying in 0.17s
[client] POST /orders attempt 2/3 failed (TimeoutError: timed out); retrying in 0.37s
   -> HTTP 201, order #3

4) SERVER DOWN: 3 failed calls in a row open the breaker
[client] GET /products attempt 1/3 failed (HTTP 503); retrying in 0.14s
[client] GET /products attempt 2/3 failed (HTTP 503); retrying in 0.36s
[client] GET /products attempt 3/3 failed (HTTP 503); giving up
   call 1: gave up (GET /products failed: HTTP 503)
[client] GET /products attempt 1/3 failed (HTTP 503); retrying in 0.10s
[client] GET /products attempt 2/3 failed (HTTP 503); retrying in 0.39s
[client] GET /products attempt 3/3 failed (HTTP 503); giving up
   call 2: gave up (GET /products failed: HTTP 503)
[client] GET /products attempt 1/3 failed (HTTP 503); retrying in 0.19s
[client] GET /products attempt 2/3 failed (HTTP 503); retrying in 0.34s
[client] GET /products attempt 3/3 failed (HTTP 503); giving up
[breaker] 3 failed call(s) -> OPEN for 3.0s
   call 3: gave up (GET /products failed: HTTP 503)
   breaker state: OPEN

5) BREAKER OPEN: next call fails fast, server is not contacted
[client] GET /products rejected: circuit is OPEN (server not contacted)
   -> rejected immediately (circuit open, not calling /products); requests that reached server: 0

6) RECOVERY: server comes back, wait for the reset timeout, trial call closes the breaker
[breaker] reset timeout over -> HALF-OPEN (one trial call allowed)
[breaker] trial call succeeded -> CLOSED (back to normal)
   -> HTTP 200; breaker state: CLOSED
```

Result. Stage 1: the call succeeds on the first attempt. Stage 2: two 503 responses are retried and the third attempt succeeds, and the server saw exactly one order for the key. Stage 3: a timeout is retried and succeeds. Stage 4: three whole calls fail (each after three attempts), the breaker opens. Stage 5: the next call is rejected immediately and the server receives no request. Stage 6: after the reset time one trial call succeeds and the breaker closes. Rate limiting is shown separately

(Figure 1) because a 429 is a deliberate answer from a healthy server and must not be treated as a failure.

1.9 Automated test suite

All 45 tests pass (Figure 4). They need only `pytest` and use temporary databases or the fake server, never real data.

File	Tests	What it covers
`test_openapi_conformance.py`	27	Spec validity; every endpoint against its schema; valid, invalid and repeated orders; idempotency; rate limit; pending-orders gauge refreshed from the database.
`test_grpc_service.py`	10	Both RPCs; every gRPC status code; idempotent replay with trailing metadata; metrics counted correctly; bounded method labels; created-orders counter excludes replays.
`test_client_reliability.py`	8	Success; retry then success; timeout without duplicate order; give up after 3 attempts; no retry for unsafe calls; breaker open, fail fast and recover; failed trial re-opens; 429 honours `Retry-After`.

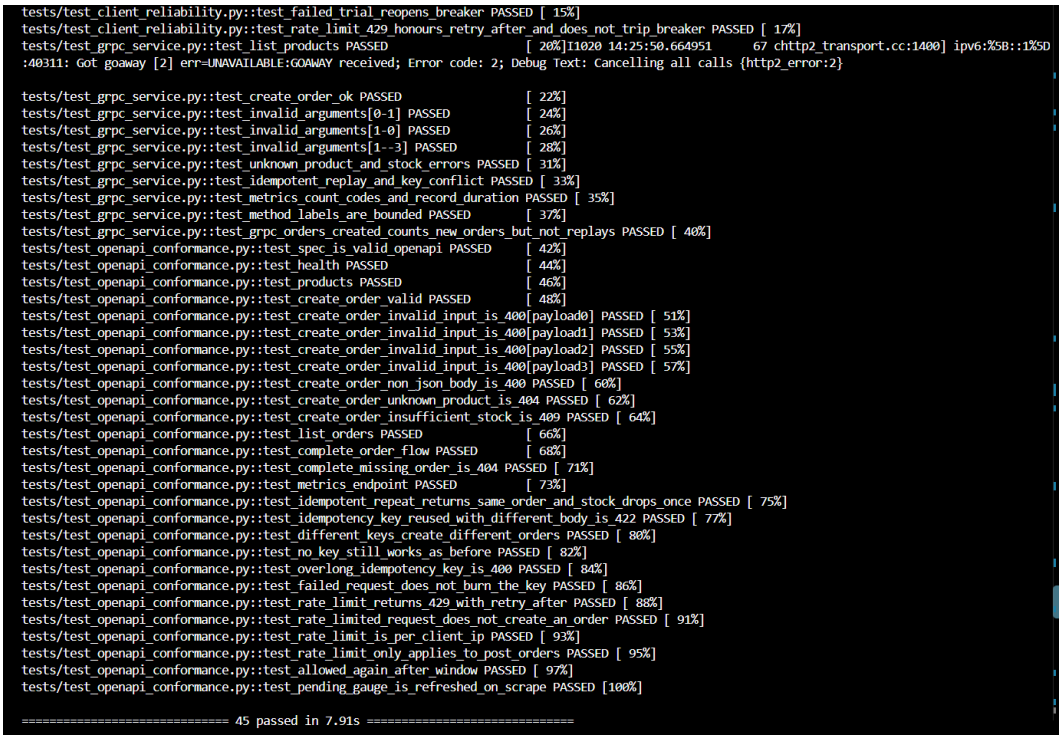


Figure 4: Full test run: 45 passed.

2 Part B: gRPC service

2.1 gRPC contract and code generation

The order operations are offered as a gRPC service. The contract is `proto/petstore.proto`: package `petstore.v1`, service `PetStore`, with two unary RPCs:

- `ListProducts(ListProductsRequest) returns (ListProductsResponse)`, the same data as `GET /products`.
- `CreateOrder(CreateOrderRequest) returns (Order)`, the same rules as `POST /orders`. The request carries `product_id`, `quantity` and an optional `idempotency_key`; the response is an `Order` with the same fields as the HTTP JSON.

The Python code is generated with grpcio-tools 1.84.0 (the Docker build and the tests run this command themselves):

```
mkdir -p grpc_gen
python -m grpc_tools.protoc -I proto --python_out=grpc_gen --grpc_python_out=grpc_gen proto/petstore.proto
```

This produces `petstore_pb2.py` (message classes, serialised to binary Protocol Buffers) and `petstore_pb2_grpc.py` (the client stub `PetStoreStub` and the server base class `PetStoreServicer`).

2.2 gRPC server, shared rules and client

The gRPC server (`grpc_server.py`) runs in its own container, `grpc-service`, on port 50051, with its metrics on port 8001. `PetStoreServicer.CreateOrder` extends the generated base class and applies the same rules as the REST endpoint: validate the arguments, hash the request, `BEGIN IMMEDIATE`, look up the idempotency key, check product and stock, insert the order, reduce stock, store the key, commit. Both services use the same SQLite database on the shared Docker volume, so they have one source of truth, and the idempotency table means the same on both. The gRPC service never calls the HTTP API.

The rules are written twice (REST and gRPC) rather than moved into a shared module; this left Assignment 1 code untouched and is listed under limitations (Section 4).

The Python client (`client/grpc_client_demo.py`) calls the service through the generated stub. A replayed key returns the original order with trailing metadata `idempotent-replayed=true`, the gRPC counterpart of the HTTP header.

2.3 Call paths

Figure 5 shows the path of the order operation over HTTP and over gRPC, and how metrics reach Grafana.

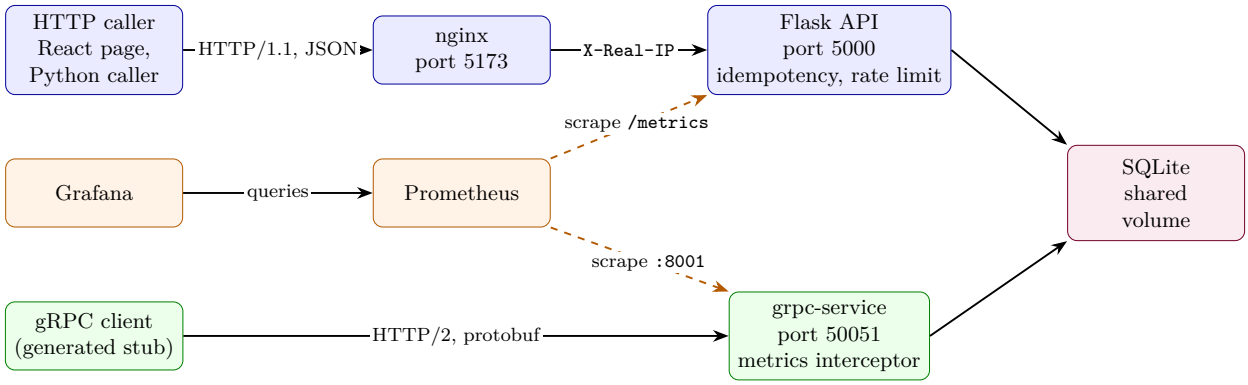


Figure 5: Call paths for placing an order over HTTP (blue) and gRPC (green). Both end in the same business rules and the same database. Dashed orange lines are Prometheus scrapes; Grafana reads from Prometheus.

HTTP path. The caller sends `POST /orders` with a JSON body and an `Idempotency-Key` over

HTTP/1.1. nginx forwards it to Flask (adding `X-Real-IP`), which applies the rate limit, validation and the transaction, then answers in JSON.

gRPC path. The client builds a `CreateOrderRequest` and calls the generated stub, which serialises it to Protocol Buffers and sends it over HTTP/2. In the `grpc-service` container the metrics interceptor wraps the call (counting, timing, in-flight), then the servicer runs the same rules against the same database.

2.4 Success and failure through both versions

The same operation, creating an order, behaves the same in both protocols; only the way errors are expressed differs. Values are illustrative; field names come from the code.

```
HTTP  POST /orders  Idempotency-Key: demo-123   {"product_id": 1, "quantity": 2}
      -> 201 {"id": 7, "product_id": 1, "product_name": "Dog Food", "quantity": 2,
              "unit_price": 12.5, "total_price": 25.0, "status": "pending", ...}
gRPC  CreateOrder(product_id=1, quantity=2, idempotency_key="demo-123")
      -> Order{id: 7, product_id: 1, product_name: "Dog Food", quantity: 2, ...}

HTTP  quantity 0          -> 400 {"error": "quantity must be a positive integer."}
gRPC  quantity 0          -> INVALID_ARGUMENT
HTTP  product 9999        -> 404 {"error": "Product not found."}
gRPC  product 9999        -> NOT_FOUND
HTTP  quantity too large  -> 409 {"error": "Not enough stock available.", "available_stock": 1}
gRPC  quantity too large  -> FAILED_PRECONDITION
```

Situation	HTTP	gRPC	Why this code
Order created	201 Created	`OK`	The request succeeded.
Replay of an earlier request	201 + `Idempotent-Replayed`	`OK` + trailing metadata	Same result as the original; flagged as a replay.
Bad product id or quantity	400 Bad Request	`INVALID_ARGUMENT`	The request itself is malformed.
Unknown product	404 Not Found	`NOT_FOUND`	The resource does not exist.
Not enough stock	409 Conflict	`FAILED_PRECONDITION`	A valid request that conflicts with the current state; retrying will not help until stock changes.
Key reused, different body	422	`ALREADY_EXISTS`	The key already belongs to another request.
Too many requests	429 + `Retry-After`	(not limited)	The rate limit was applied to the HTTP API only, as the brief asks.

HTTP puts its status in the response status line with details in the JSON body; gRPC runs over HTTP/2 and returns its own status code and message in the trailers, which the client receives as a `grpc.RpcError` with `code()` and `details()`. gRPC has fewer, more general codes, so one gRPC code can cover several HTTP ones.

Repeatable success and failure calls. `client/workload.py` sends seeded, realistic traffic (mostly normal orders, some retries with the same key, and deliberate failures), then reads the stock before and after and checks that the drop equals the quantity of orders actually created for every product. Both runs ended with `RESULT: PASS`.

Run	Requests	Outcomes
gRPC, seed 2026, 60 per minute	60	47 new orders OK, 5 retries OK (replayed), 4 `INVALID_ARGUMENT`, 4 `NOT_FOUND`; stock reconciled for all 6 products.
HTTP, seed 2026, 9 per minute	20	16 created (201), 3 bad quantity (400), 1 unknown product (404); stock reconciled for all 6 products.
gRPC burst, 16 threads for 30 s	7 967 calls	265 calls per second, 10 real orders (capped), 0 errors, 16 calls in flight seen by the client.

New orders versus replays. Of the 52 successful gRPC calls, 47 created a new order and 5 were deliberate retries with the same key. The retries returned the original order, took no extra stock, and are not counted by `petstore_grpc_orders_created_total`; this is why stock fell by exactly the quantity of the 47 new orders. The burst is a short, capped concurrent traffic demonstration (at most 10 real orders) that makes the in-flight gauge visible in Grafana; it is not a performance benchmark.

3 Part C: Metrics

3.1 Audit of the Assignment 1 metrics

Assignment 2 added a second process (gRPC) that shares one database with Flask, which made some Assignment 1 metrics incomplete or wrong. Each was checked against the new design:

Metric (type)	What it shows	Verdict	Action
`petstore_http_request_duration_seconds` (Histogram; method, route, status_code)	HTTP latency, calls and errors (its `_count` series), including 429	Useful, but buckets too coarse for fast calls	Keep; add finer buckets between 5 and 100 ms.
`petstore_orders_total` (Counter)	Orders created	Wrong for the new design: gRPC orders not counted	Keep; add a gRPC counterpart and sum both on the dashboard.
`petstore_pending_orders` (Gauge)	Orders waiting to be completed	Wrong: only updated by Flask, so gRPC orders left it stale	Fix: recompute from SQLite on every `/metrics` scrape.
Node exporter (CPU, memory, disk)	Docker host resources	Useful context	Keep.

Gaps and additions.

- **No gRPC metrics:** added calls by method and status, a duration histogram and an in-flight gauge, measured at the server, plus a Prometheus job for the gRPC service.
- **Histogram resolution:** with the original buckets, p50 and p95 fell into the same bucket and looked equal; denser buckets were added (HTTP and gRPC).
- **No reason for rejected orders:** an error rate does not say why, so rejections are counted by reason.
- **Stale shared state:** the pending-orders gauge now reads the database (the "other missing project metric").

- **Labels:** every new label has a small, fixed set of values; product ids, order ids and idempotency keys are never labels.

3.2 New and changed metrics

All metrics are measured at the **server**: in the gRPC interceptor and the Flask code. The clients do not export metrics.

Metric	Purpose	Type	Unit	Labels
`grpc_server_handled_total`	Calls, and error rate, by method and status	Counter	calls	`grpc_service`, `grpc_method`, `grpc_code`
`grpc_server_handling_seconds`	Server time per call (p50, p95, distribution); buckets 5, 10, 15, 20, 30, 40, 50, 75, 100, 250, 500 ms, 1, 2.5, 5 s	Histogram	seconds	same three
`grpc_server_in_flight_requests`	Calls being handled right now	Gauge	calls	`grpc_service`, `grpc_method`
`petstore_grpc_orders_rejected_total`	Why orders were refused	Counter	orders	`reason`: four fixed values
`petstore_grpc_orders_created_total`	New gRPC orders (replays excluded)	Counter	orders	none
`petstore_http_request_duration_seconds` (changed)	HTTP side of the comparison, finer buckets	Histogram	seconds	`method`, `route`, `status_code`
`petstore_pending_orders` (changed)	Pending orders, recomputed at each scrape	Gauge	orders	none

Metric	Code location
`grpc_server_handled_total`, `grpc_server_handling_seconds`, `grpc_server_in_flight_requests`	`grpc_server.py`, `MetricsInterceptor` (updated for every RPC in one place)
`petstore_grpc_orders_rejected_total`, `petstore_grpc_orders_created_total`	`grpc_server.py`, `CreateOrder`
`petstore_http_request_duration_seconds`	`metrics.py` (buckets) and the Flask request hooks in `app.py`
`petstore_pending_orders`	`app.py`, `update_pending_orders_gauge`, called from `/metrics`

3.3 Queries

Rates use a 1-minute window; error rates, quantiles and histograms use a 5-minute window. The HTTP panels use the existing duration histogram.

```
# gRPC calls per second by method and status
sum by (grpc_method, grpc_code) (rate(grpc_server_handled_total[1m]))

# gRPC failed calls per second by method and status
sum by (grpc_method, grpc_code) (rate(grpc_server_handled_total{grpc_code!="OK"}[5m]))

# gRPC error rate (%) per method
100 * sum by (grpc_method) (rate(grpc_server_handled_total{grpc_code!="OK"}[5m]))
```

```
    / sum by (grpc_method) (rate(grpc_server_handled_total[5m]))

# gRPC duration p50 and p95 over a 5-minute window
histogram_quantile(0.50, sum by (le, grpc_method) (rate(grpc_server_handling_seconds_bucket[5m])))
histogram_quantile(0.95, sum by (le, grpc_method) (rate(grpc_server_handling_seconds_bucket[5m])))

# gRPC in flight
max by (grpc_method) (grpc_server_in_flight_requests)

# gRPC rejected orders by reason, and new orders created
sum by (reason) (increase(petstore_grpc_orders_rejected_total[5m]))
sum(increase(petstore_grpc_orders_created_total[5m]))

# HTTP comparison: calls and errors by status for POST /orders
sum by (status_code) (rate(petstore_http_request_duration_seconds_count{route="/orders",method="POST"}[1m])
    )
sum by (status_code) (rate(petstore_http_request_duration_seconds_count{route="/orders",method="POST",
    status_code!~"2.."}[5m]))
histogram_quantile(0.95, sum by (le) (rate(petstore_http_request_duration_seconds_bucket{route="/orders",
    method="POST"}[5m])))
```

3.4 Design choices

Where duration is measured. At the server: the gRPC timer runs from when the call reaches the interceptor until the handler returns, and the HTTP timer covers the request handling in Flask. Network and client waiting are not included, so both versions are compared on the server's own work. The workload script prints client-side times for comparison; they are larger (about 19 ms for gRPC and 34 ms for HTTP on average in the runs above) because they add connection and serialisation time.

Buckets. Real server times were about 2–45 ms. The original buckets had one wide bucket around 10–25 ms, so p50 and p95 both landed in it and looked almost equal. Buckets at 15, 20, 30, 40 and 75 ms separated them, and coarse buckets continue up to 5 s so a slow server still lands in a bucket rather than in `+Inf`.

In-flight calls instead of message size. The assignment is about load and reliability. In-flight calls show how busy the server is and would reveal calls piling up if the database slowed. A request here is a few tens of bytes, so a message-size metric would be an almost flat line.

Other missing metric. The pending-orders gauge was the most important gap: it lived in Flask's memory while gRPC could change orders, so it is now computed from the shared database at each scrape. The rejection-reason counter and the gRPC created-orders counter were added for the same reason: the audit showed those events were invisible.

Bounded labels. `grpc_method` is collapsed to the two known method names or `unknown`, so a caller cannot create unlimited series by calling made-up methods (tested); `grpc_code` is one of the gRPC status names; `reason` has four values. No label contains an id, key or address.

3.5 Prometheus and Grafana results

Prometheus scrapes the Flask backend (`backend:5000/metrics`), the gRPC service (`grpc-service:8001`) and the node exporter (Figure 6). The dashboard row "Create an Order: HTTP (POST /orders) vs gRPC (CreateOrder)" shows the HTTP operation beside the gRPC operation (Figures 7 and 8). Traffic came from the seeded workloads and one capped burst described in Part B.

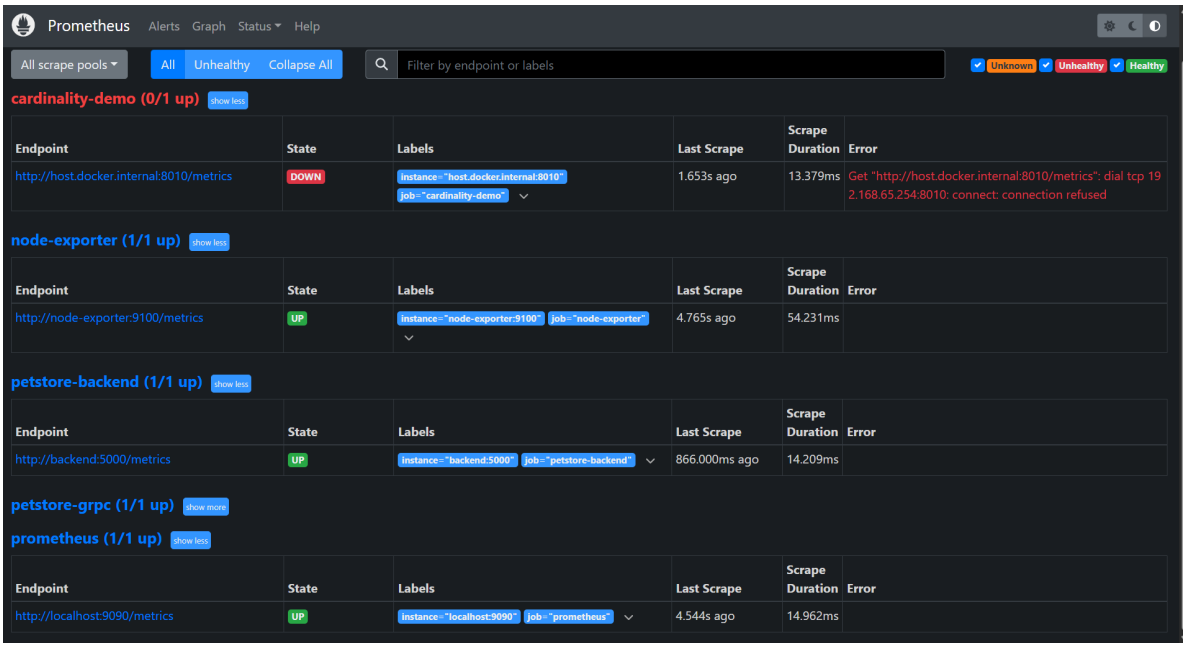


Figure 6: Prometheus targets: the `petstore-grpc` job is up beside the Flask job.

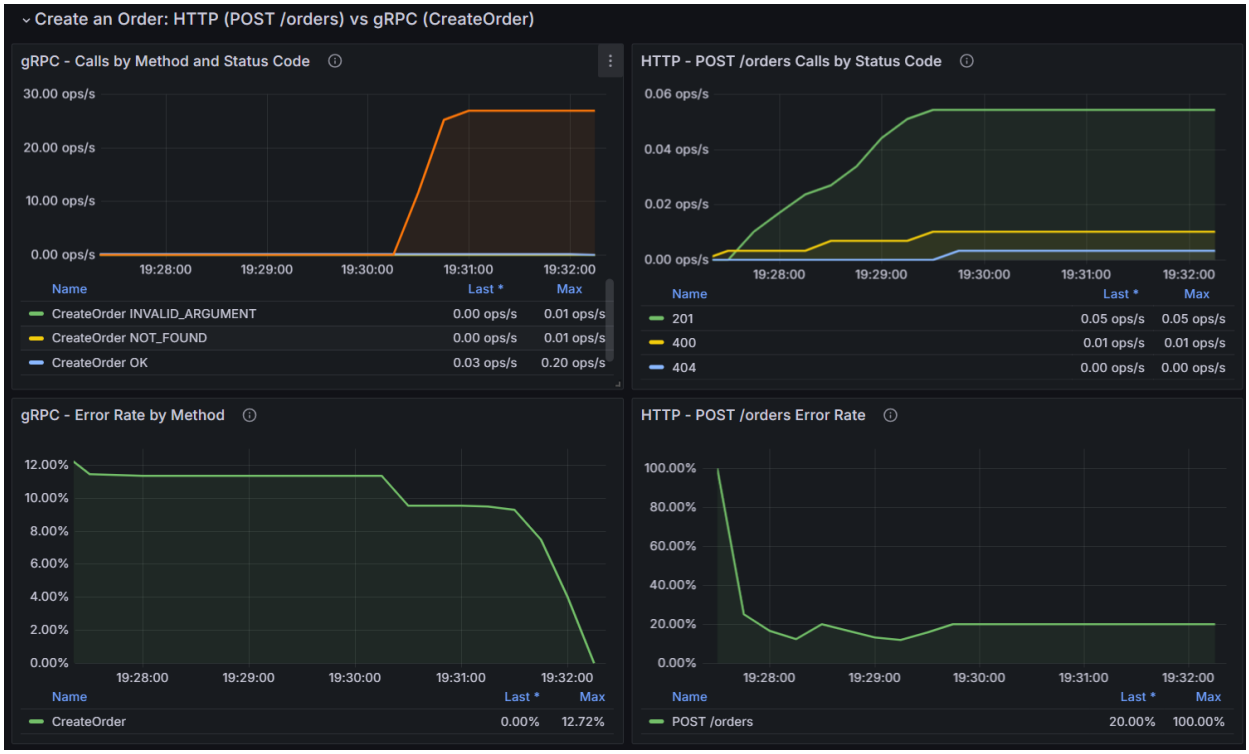


Figure 7: Calls per second by status and error rate, HTTP and gRPC side by side.



Figure 8: Duration p50 and p95 (5-minute window), gRPC calls in flight, and orders rejected by reason.

Reading the panels.

- **Latency:** before the burst, server-side gRPC p50 was about 10–14 ms and p95 about 25–45 ms; HTTP `POST /orders` p50 was about 14 ms and p95 about 26 ms. The two paths do the same database work, so they are close; the finer buckets make p50 and p95 visibly different.
- **Burst and in-flight:** during the 30-second burst gRPC `CreateOrder` p95 rose to about 175 ms and p50 to 45 ms, and `ListProducts` calls in flight peaked at 8. With one caller the gauge stays near 0, because calls finish in milliseconds and Prometheus samples every 5 seconds; real concurrency is needed to see it.
- **Error rate:** the gRPC error rate sits near 12% because the workload deliberately includes bad and unknown-product calls (8 of 60). Percentages are noisy on small samples (one failure in a few calls is a large share, and the first request in a window can read 100%), and the burst dilutes the gRPC rate with thousands of successful reads.
- **Rejections:** the rejected-by-reason panel separates bad arguments, unknown products and stock problems, which the error rate alone cannot.

The business overview and HTTP panels (Figures 9 and 10) now include gRPC orders in the totals, because the gRPC created-orders counter is summed with the Assignment 1 counter.

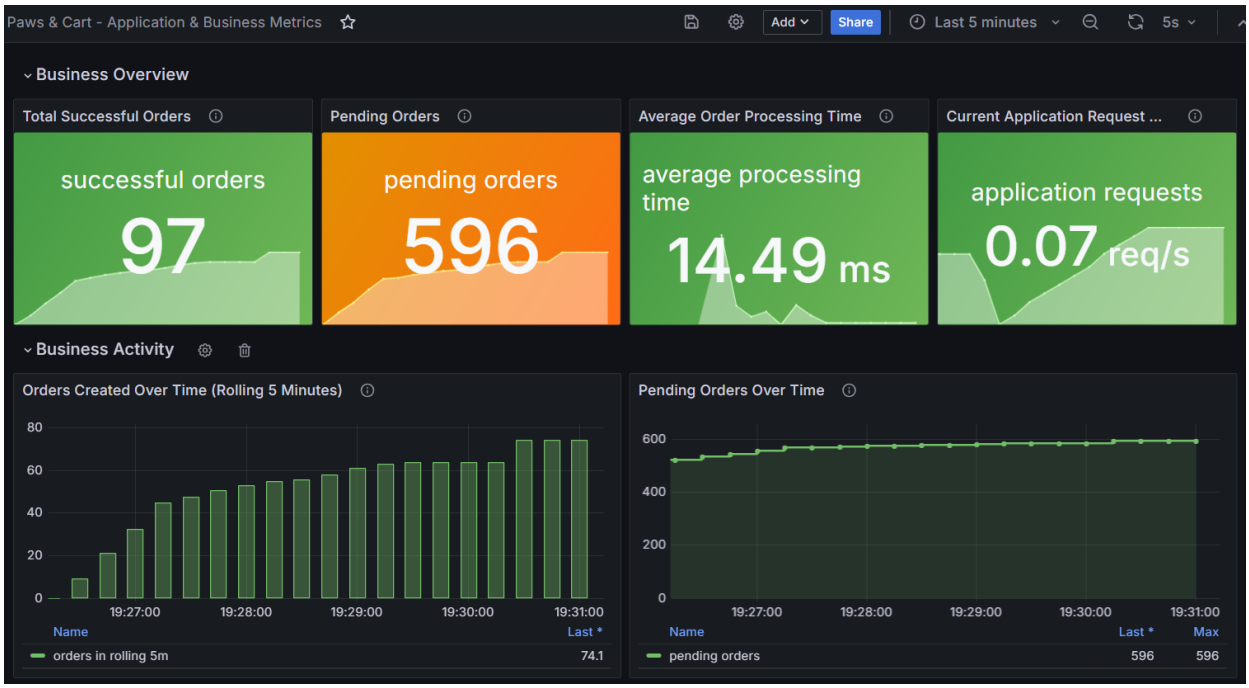


Figure 9: Business overview panels, including orders from both services.



Figure 10: HTTP traffic and latency panels.

4 Limitations and design choices

- The rate limiter keeps its counts in memory, so it is correct for one Flask process only; several instances would each allow 10 requests.
- The gRPC service has no rate limiter; the brief asks for one on the HTTP API.

- The REST and gRPC services share one SQLite file through a Docker volume; this is fine at this scale, but writes are serialised.
- The order rules are written twice (REST and gRPC). This keeps Assignment 1 code untouched, but a rule change must be made in both places.
- The circuit breaker's half-open state lets calls through after the reset time without enforcing a single trial across threads; with one calling thread, as in the demo, exactly one trial happens.
- Idempotency keys are never deleted; a production system would expire them after a retention period.
- Prometheus samples every 5 seconds, so the in-flight gauge only shows under sustained concurrent traffic. Error-rate panels are noisy on small samples.
- The gRPC channel is insecure (no TLS), acceptable inside a local Docker network for this assignment.